

Happy Weddings: A Database Systems Case Study

Fabio Cirullo (841289)

Contents

1	Requirements Text	2
2	Requirements Collection and Analysis	2
2.1	Level of Abstraction	3
2.2	Standardized sentence structure	3
2.3	Identification of Homonyms and Synonyms	4
2.4	Glossary of Terms	5
2.5	Sentences grouped by Concept	6
3	Conceptual Design	7
3.1	Design Strategy and Skeleton Schema	7
3.2	Final ER Schema	7
3.3	Business Rules	7
4	Logical Design	9
4.1	Volume and Operation Tables	9
4.2	Redundancy Analysis	10
4.3	Access Table	11
4.4	Logical Schema	12
5	Physical Design	14
5.1	Types and Tables definition	14
5.2	Triggers	19
5.3	Procedures and Functions	22
5.4	Query Optimization	24
6	Data Warehouse	27
6.1	Facts	27
6.2	Measures	27
6.3	Dimensions	27
6.4	The Constellation Schema	28
7	Web Application	29
7.1	Architecture	29
7.2	Connection Management	29
7.3	Accessing the Object-Relational Schema	30
7.4	Transactions and Integrity	34

1 Requirements Text

The company **HappyWeddings (HW)**, specialized in wedding planning, offers its clients a wide range of services, for each of which the following is known: a brief description, the minimum and maximum cost of organizing the service, and an indication of how far in advance of the wedding the service must be ordered. The services offered can be divided into four categories: clothing, wedding registry stores, flowers, and catering. The clothing section is in turn divided into bridal clothing and groom clothing, and for each outfit the following is specified: the type of material used, the color, the various models of shoes that can be matched with it, the list of stores to go to for fittings, and, only in the case of groom clothing, the list of accessories that can be matched with it. For each accessory, the type (e.g., tie, cufflinks), the material, and the cost are known. As for the wedding registry stores section, the following is provided: the store address, the phone number, and the type of store. The flowers section includes the type of arrangement (e.g., flowers for the church, for the town hall, bouquet) and the list of flowers that make up the arrangement. Flowers may be available in different shades and only during certain periods of the year. Note that the same flower may be part of several arrangements. For catering, various combinations are proposed, for each of which the following is known: the menu (a menu consists of a description of the dishes provided, with an indication of whether it is an appetizer, first course, second course, or dessert, and a list of wines), and the name and address of the restaurant. HW's clients can book one or more services by filling out an order form, on which the following information is reported: the client's first name, last name, and phone contact; the list of requested services; the wedding date; and finally the date the form was filled out. HW's clients may order several services at different times. After the wedding has taken place, the client receives an invoice, which reports the date it was drawn up, the list of services actually provided along with the final cost for each of them, and the payment method.

Operations:

1. Insert a new client into the company, including first name, last name, and phone contact – 15 times per day.
2. Given a specific catering service, print the name and address of its restaurant, together with all the menus it proposes; for each menu, print the list of dishes (each with its type and description) and the list of wines – 60 times per day.
3. Insert a new order form filled out by a client, including the wedding date, the date the form was filled out, and the list of requested services – 100 times per day.
4. Given a specific client, print the list of all services he/she has requested (across all of his/her order forms), including each service's description, minimum cost, maximum cost, and category (clothing, wedding registry store, flowers, or catering) – 120 times per day.

Taking into account that there are 500 services offered (20% clothing, 25% wedding registry stores, 30% flowers, 25% catering); that each groom outfit can be matched with an average of 4 accessories; that each flower arrangement is composed of an average of 6 base flowers, and the same base flower appears on average in 3 arrangements; that each catering service proposes an average of 3 menus, and each menu lists an average of 5 wines and 8 dishes; the company also has 1,500 clients, each of whom has filled out an average of 4 order forms in the period considered, and each order form requests an average of 3 services.

2 Requirements Collection and Analysis

Based on the standard methodology for conceptual database design, the requirements analysis phase does not aim at producing the conceptual schema directly. Its purpose is to take the natural language specification - which is typically ambiguous, redundant, and unevenly detailed - and turn it into a clarified and organized text that can be mapped onto ER constructs in a systematic way. This is done by following some general rules:

1. choosing the correct level of abstraction for each concept;

2. standardizing the structure of the sentences;
3. identifying homonyms and synonyms and unifying the terminology;
4. building a glossary of terms;
5. grouping the sentences by concept.

2.1 Level of Abstraction

The original text mixes several levels of abstraction, and some terms are either too generic or left implicit. The following adjustments were made:

- "a brief description" of a service is kept as a textual attribute, since no further structure is given for it.
- "the minimum and maximum cost of organizing the service" is kept as two separate attributes, since no further structure is given for them.
- "an indication of how far in advance... the service must be ordered" is made explicit as a numeric lead time (expressed in days), attached to the service.
- "the type of material used" for an outfit and "the material" of an accessory refer to the same concept applied to two different entities; the abstraction level (a simple descriptive attribute) is appropriate in both cases and does not need to be turned into an autonomous entity, since no further properties of "material" are requested.
- "the various models of shoes" and "the list of stores to go to for fittings" are kept as simple multivalued attributes of the outfit, since the text requests no further properties for shoe models or fitting stores beyond their identifying name.
- "flowers may be available in different shades and only during certain periods of the year" is made explicit as two separate attributes (shade and availability period) attached to the flower entity, rather than left as a vague qualitative note.

2.2 Standardized sentence structure

Each requirement is rewritten using a uniform syntactic pattern: "For each <concept>, we represent <properties>" or "A <concept> is associated with <related concept>." Rewriting the specification this way, before any decomposition, gives:

- For each service, we represent a brief description, the minimum and maximum cost of organizing the service, and an indication of how far in advance of the wedding the service must be ordered.
- The services offered can be divided into four categories: clothing, wedding registry stores, flowers, and catering.
- The clothing section is in turn divided into bridal clothing and groom clothing.
- For each outfit, we represent the type of material used, the color, the various models of shoes that can be matched with it, and the list of stores to go to for fittings.
- For each accessory, we represent the type (e.g., tie, cufflinks), the material, and the cost.
- For each wedding registry store, we represent the store address, the phone number, and the type of store.
- For each flower arrangement, we represent the type of arrangement (e.g., flowers for the church, for the town hall, bouquet) and the list of flowers that make up the arrangement.
- For each flower, we represent the shade and the availability period.
- For each catering combination, we represent the menu, and the name and address of the restaurant.

- For each menu, we represent a description of the dishes provided, with an indication of whether it is an appetizer, first course, second course, or dessert, and a list of wines.
- For each order form, we represent the client's first name, last name, and phone contact; the list of requested services; the wedding date; and the date the form was filled out.
- For each invoice, we represent the date it was drawn up, the list of services actually provided along with the final cost for each of them, and the payment method.

2.3 Identification of Homonyms and Synonyms

Several terms are reused in the original text with different meanings (homonyms), while others describe the same concept with different words (synonyms). These must be unified before building the glossary.

Homonyms

- "store" is used both for the fitting locations associated with a clothing outfit and for the wedding registry stores. These are two unrelated concepts: fitting stores are points of reference for trying on outfits, with no properties beyond identification, while wedding registry stores are catalog entries with their own address, phone number, and type. They are renamed Fitting Store and Wedding Registry Store respectively.
- "type" appears with four meanings: type of accessory (tie, cufflinks), type of wedding registry store, type of flower arrangement, and type of dish (appetizer, first course, etc.). They are renamed with a specific attribute name: Accessory Type, Store Type, Arrangement Type, Dish Type.
- "cost" appears with three meanings: the minimum or maximum cost of a service, the cost of an accessory, and the final cost of a service on the invoice. They are renamed Minimum Cost / Maximum Cost (Service), Accessory Cost, and Final Cost (Invoice line).
- "date" appears three times with different referents: the wedding date, the date the order form was filled out, and the date the invoice was drawn up. They are renamed Wedding Date, Filing Date, and Issue Date.
- "list of services" appears both in the order form (services requested) and in the invoice (services actually provided); they are not the same association and are kept distinct as Requested Services and Provided Services.
- "flowers" is used both as the name of one of the four service categories and as the individual flower species that make up an arrangement. They are two distinct concepts at different levels: the category identifies a type of service offered by HW, while the flower identifies an item used within that service. The category is renamed Flower Service, while the individual item keeps the name Flower.

Synonyms

- "outfit" and "clothing" are used interchangeably in the original text; they are unified under the term Clothing.

2.4 Glossary of Terms

Term	Description	Synonyms	Connections
Client	Person who books wedding services from HW	—	Order Form, Invoice
Order Form	Document through which a client requests one or more services	Order	Client, Service
Invoice	Document issued after the wedding, listing provided services, final costs, and payment method	—	Client, Service
Service	General catalog item offered by HW, belonging to one of four categories	—	Clothing, Wedding Registry Store, Flower Service, Catering Service, Order Form, Invoice
Clothing	Bridal or groom clothing item, specialization of Service	Outfit	Fitting Store, Accessory
Accessory	Item that can be matched with a groom outfit	—	Clothing
Fitting Store	Location where a client can try on an outfit	—	Clothing
Wedding Registry Store	Shop where a couple can register a wedding gift list, specialization of Service	Registry Store	—
Flower Service	Combination of flowers proposed for a specific use (church, town hall, bouquet), specialization of Service	Flower Arrangement	Flower
Flower	Individual flower species that can be part of one or more flower services, with available shades and periods	—	Flower Service
Catering Service	Combination proposed for catering, specialization of Service	Combination	Menu, Restaurant
Menu	Set of dishes and wines proposed within a catering service	—	Catering Service, Dish, Wine
Dish	Individual course of a menu, classified by type	—	Menu
Wine	Wine associated with a menu	—	Menu
Restaurant	Venue associated with a catering service, identified by name and address	—	Catering Service

Table 1: Glossary of terms

2.5 Sentences grouped by Concept

Following the last general rule, the standardized sentences are reorganized by concept, repeating shared information where necessary, so that each group can later be examined independently during conceptual design.

General services. For each service, we represent a description, a minimum cost, a maximum cost, and the required advance-booking time. Every service belongs to exactly one of four categories: clothing, wedding registry stores, flowers, or catering.

Clothing. For each outfit (bridal or groom), we represent the material, the color, the list of matching shoe models, and the list of fitting stores. For each groom outfit only, we additionally represent the list of matching accessories. For each accessory, we represent an accessory type, a material, and a cost.

Wedding registry stores. For each wedding registry store, we represent an address, a phone number, and a store type.

Flowers. For each flower service, we represent the arrangement type and the list of flowers that make up the arrangement. For each flower, we represent the shade and the availability period. The same flower can be part of several flower services.

Catering. For each catering service, we represent the list of menus it proposes, and the name and address of the restaurant. For each menu, we represent the list of dishes and the list of wines. For each dish, we represent a description and a dish type (appetizer, first course, second course, or dessert).

Clients and order forms. For each client, we represent a first name, a last name, and a phone contact. A client can order several services at different times, by filling out one or more order forms. For each order form, we represent the requested services, the wedding date, and the filing date.

Invoices. After the wedding, a client receives an invoice. For each invoice, we represent the issue date, the payment method, and, for each provided service, its final cost.

3 Conceptual Design

3.1 Design Strategy and Skeleton Schema

The conceptual design follows an hybrid approach. First, a skeleton schema is defined, including only the main concepts obtained by grouping the requirements. This initial schema is then progressively refined until the final ER schema is obtained. Figure 1 shows the skeleton schema, which includes Client, OrderForm, Invoice, Service, and the four service categories. In this schema, attributes, relationship names, cardinalities, and the more detailed internal structure are not included.

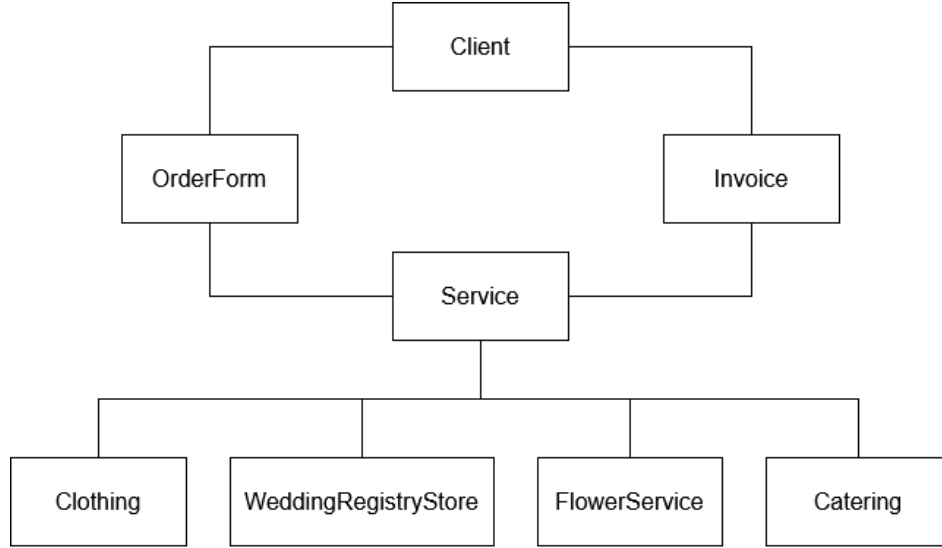


Figure 1: Skeleton Schema

3.2 Final ER Schema

The concepts in the skeleton schema are subsequently expanded to obtain the final conceptual schema shown in Figure 2. For example, Clothing is divided into its relevant elements, including material, color, shoe models, fitting stores, the Bridal/Groom specialization, and Accessory. FlowerService is expanded with the arrangement type and the Flower concept. Catering includes Restaurant, Menu, Dish, and Wine. WeddingRegistryStore is also described through its address, phone number, and store type. Finally, the attributes and identifiers of Client, OrderForm, Invoice, and Service are added.

3.3 Business Rules

The ER model can only express structural constraints. The following business rules complete the schema documentation, covering constraints and design decisions that cannot be represented graphically:

- **(BR1)** A client can organize different weddings over time. Each wedding is identified by the client and the corresponding WeddingDate. For the same wedding, the client can submit more than one order form. After the wedding, a single invoice is issued for that client and includes the services provided through all the order forms referring to that WeddingDate.
- **(BR2)** The FilingDate of an order form must precede the WeddingDate. The same applies to invoices: IssueDate must not precede WeddingDate, since an invoice is issued only after the wedding.
- **(BR3)** For every service requested in an order form, the interval between the order form's FilingDate and the WeddingDate must be greater than or equal to the LeadTime specified for that service.
- **(BR4)** In every service, MinCost must not exceed MaxCost.

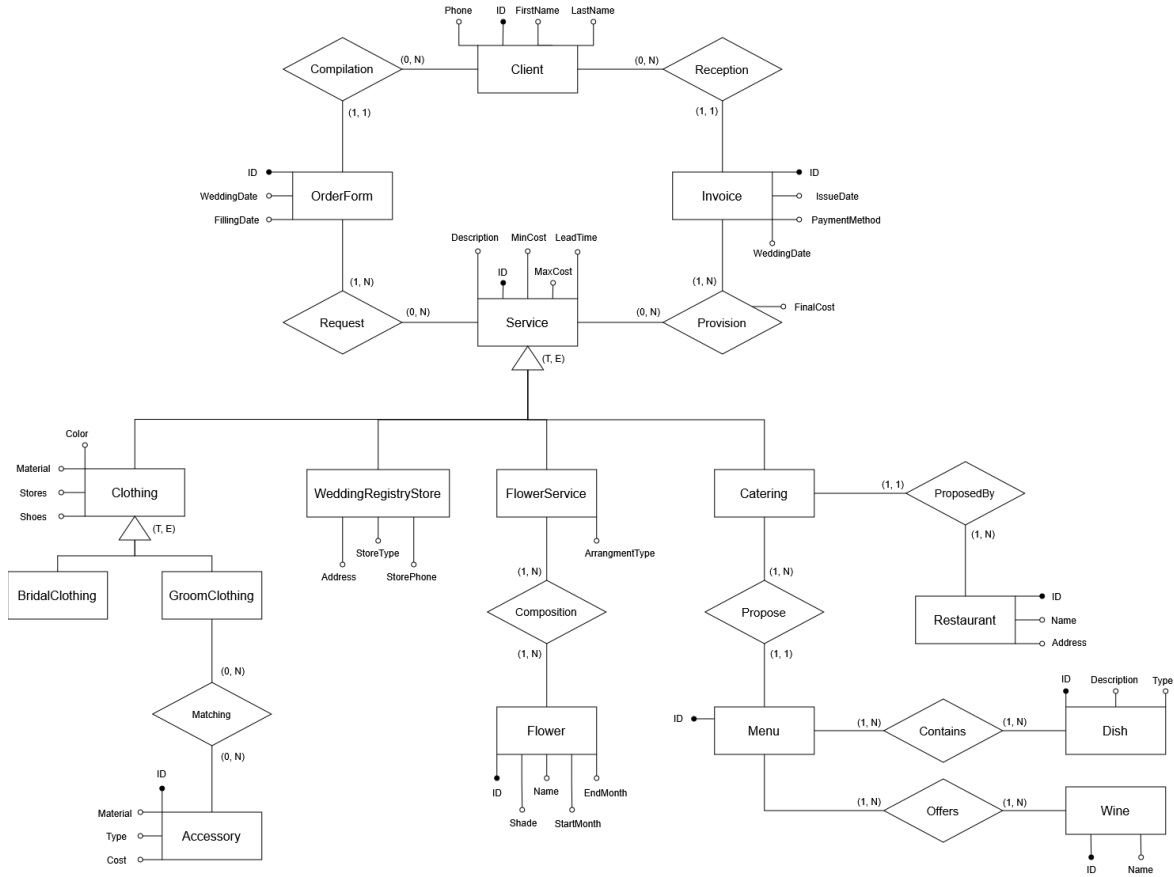


Figure 2: Final ER Schema

- **(BR5)** The FinalCost associated with a service in the Provision relationship has to be between that service's MinCost and MaxCost.
- **(BR6)** Type attribute in the Dish entity can only take one of the four values specified in the requirements: appetizer, first course, second course, or dessert.
- **(BR7)** StartMonth and EndMonth must both be valid months in the Flower entity, so their values must range from 1 to 12. A period with StartMonth greater than EndMonth is interpreted as spanning the turn of the year (e.g., from November to February).
- **(BR8)** For every invoice, WeddingDate must coincide with the WeddingDate of at least one order form filled out by the same client, so that every invoice refers to an existing wedding.

4 Logical Design

Following the conceptual design, the logical design converts the final ER schema into the corresponding relational (or object-relational) structures. The main reference for this phase is the workload described in the requirements, which consists of four operations with different expected frequencies. These operations are taken into account when calculating the volumes and accesses and when deciding whether any changes to the conceptual schema are needed.

- **Operation 1:** Insert a new client into the company, including first name, last name, and phone contact - 15 times per day.
- **Operation 2:** Given a specific catering service, print the name and address of its restaurant, together with all the menus it proposes; for each menu, print the list of dishes (each with its type and description) and the list of wines - 60 times per day.
- **Operation 3:** Insert a new order form filled out by a client, including the wedding date, the date the form was filled out, and the list of requested services - 100 times per day.
- **Operation 4:** Given a specific client, print the list of all services he/she has requested (across all of his/her order forms), including each service's description, minimum cost, maximum cost, and category (clothing, wedding registry store, flowers, or catering) - 120 times per day.

4.1 Volume and Operation Tables

The population of the entities and relationships can generally be obtained from the numerical information provided in the specification. However, some values cannot be determined directly, so a few assumptions are needed:

- The split of Clothing into BridalClothing and GroomClothing is not given: a 50/50 split is assumed.
- Whenever only one direction of a M:N relationship's average is given (e.g., Accessory per Groom-Clothing, Dish and Wine per Menu), the reverse average is not specified. In these cases, we assume that the related entities are not reused, so that each entity is associated, on average, with only one instance on the other side. Under this assumption, the population of the entity is the same as that of the corresponding relationship. Flower is the only case where both directions are given (6 flowers per arrangement; the same flower appears, on average, in 3 arrangements), so its population is computed exactly, without this assumption.

There is no corresponding information for Invoice and its relationships, Reception and Provision, so their volumes cannot be calculated from the available data. This does not affect the subsequent calculations because none of the four specified operations accesses Invoice.

Concept	Type	Volume	Notes
Client	Entity	1 500	given
OrderForm	Entity	6 000	$1\,500 \times 4$ order forms per client
Invoice	Entity	—	not specified in the data; not required by the operations
Service	Entity	500	given
Clothing	Entity	100	$20\% \times 500$
BridalClothing	Entity	50	assumption: even split of Clothing
GroomClothing	Entity	50	assumption: even split of Clothing
Accessory	Entity	200	50×4 ; assumption: no reuse across outfits
WeddingRegistryStore	Entity	125	$25\% \times 500$
FlowerService	Entity	150	$30\% \times 500$
Flower	Entity	300	$(150 \times 6)/3$
Catering	Entity	125	$25\% \times 500$
Restaurant	Entity	125	assumption: no reuse across catering services
Menu	Entity	375	125×3
Dish	Entity	3 000	375×8 ; assumption: no reuse across menus
Wine	Entity	1 875	375×5 ; assumption: no reuse across menus
Compilation	Relationship	6 000	one instance per OrderForm
Reception	Relationship	—	not specified in the data
Request	Relationship	18 000	$6\,000 \times 3$ services per order form
Provision	Relationship	—	not specified in the data
Matching	Relationship	200	50×4
Composition	Relationship	900	150×6
ProposedBy	Relationship	125	one instance per CateringService
Propose	Relationship	375	one instance per Menu
Contains	Relationship	3 000	375×8
Offers	Relationship	1 875	375×5

Table 2: Table of volumes

Op.	Description	Type	Frequency
Op1	Insert a new client	Interactive	15/day
Op2	Print restaurant, menus, dishes and wines for a given catering service	Interactive	60/day
Op3	Insert a new order form with its requested services	Interactive	100/day
Op4	Print all requested services (with category) for a given client	Interactive	120/day

Table 3: Table of operations

4.2 Redundancy Analysis

Examining the final ER schema for redundancies, the following considerations apply:

- The schema contains one cycle: Client – Compilation – OrderForm – Request – Service – Provision – Invoice – Reception – Client. A cycle is a potential source of redundancy only if one of its relationships can be derived by composing the others. In this case, Compilation and Request identify the services requested by a client, while Provision, through Reception, concerns the services that were actually provided and invoiced. These are different pieces of information, and Provision also stores FinalCost, which cannot be obtained from the other relationships. Reception is also necessary because it is the relationship that connects an invoice to its client. Therefore, none of the relationships in the cycle can be obtained from the others. The cycle therefore represents independent facts and is not redundant;
- No attribute can be computed from other attributes or from a relationship’s cardinality. For instance, FinalCost is an actual value within the [MinCost, MaxCost] range of a service and cannot be calculated from those two values. Similarly, the WeddingDate associated with an invoice is needed to identify which wedding of a given client the invoice concerns.

- The schema does not contain derived or aggregate attributes, such as counters or totals that could be obtained by counting or aggregating relationship occurrences.

The conceptual schema is therefore free of redundancies, and no restructuring is required at this step.

4.3 Access Table

For each of the four operations, the access table reports the ER constructs involved, specifying whether each one is an entity or a relationship, how many instances are accessed on average during an execution, whether the access is a read or a write, and what the operation does with that construct. The average multiplicities, such as the number of order forms per client and the number of services per order form, are those reported in the Table of Volumes.

Op1 — Insert a new client (15/day)				
Concept	Concept Type	Accesses	Access Type	Description
Client	Entity	1	Write	Insert the new client tuple

Table 4: Access table — Operation 1

Total daily accesses for Op1: 15 write/day, 0 read/day.

Op2 — Print restaurant, menus, dishes and wines of a catering service (60/day)				
Concept	Concept Type	Accesses	Access Type	Description
Catering	Entity	1	Read	Access the given catering service
ProposedBy	Relationship	1	Read	Retrieve the link to its restaurant
Restaurant	Entity	1	Read	Retrieve restaurant name and address
Propose	Relationship	3	Read	Retrieve the links to the menus it proposes
Menu	Entity	3	Read	Access each proposed menu
Contains	Relationship	24	Read	Retrieve the links to the dishes of each menu
Dish	Entity	24	Read	Retrieve type and description of each dish
Offers	Relationship	15	Read	Retrieve the links to the wines of each menu
Wine	Entity	15	Read	Retrieve each wine

Table 5: Access table — Operation 2

Total daily accesses for Op2: 0 write/day, 5,220 read/day (87 x60).

Op3 — Insert a new order form (100/day)				
Concept	Concept Type	Accesses	Access Type	Description
Client	Entity	1	Read	Access the client filling out the form
OrderForm	Entity	1	Write	Insert the new order form tuple
Compilation	Relationship	1	Write	Link the new order form to the client
Service	Entity	3	Read	Access each requested service
Request	Relationship	3	Write	Link the order form to each requested service

Table 6: Access table — Operation 3

Total daily accesses for Op3: 500 write/day (5 x 100), 400 read/day (4 x 100).

Op4 — Print all requested services of a client (120/day)				
Concept	Concept Type	Accesses	Access Type	Description
Client	Entity	1	Read	Access the given client
Compilation	Relationship	4	Read	Retrieve the links to the client's order forms
OrderForm	Entity	4	Read	Access each order form of the client
Request	Relationship	12	Read	Retrieve the links to the requested services
Service	Entity	12	Read	Retrieve description, minimum cost, and maximum cost
Clothing	Entity	2.4	Read	Determine category for services belonging to Clothing (20% of 12)
WeddingRegistryStore	Entity	3	Read	Determine category for services belonging to WeddingRegistryStore (25% of 12)
FlowerService	Entity	3.6	Read	Determine category for services belonging to FlowerService (30% of 12)
Catering	Entity	3	Read	Determine category for services belonging to Catering (25% of 12)

Table 7: Access table — Operation 4

Total daily accesses for Op4: 0 write/day, 5,400 read/day (45 x 120).

4.4 Logical Schema

The final ER schema was translated into an object-relational logical schema, shown in Figure 3. The main choices made during this mapping are described below, including the cases where the logical schema differs from the conceptual one.

Identification. The entities that required an identifier at the conceptual level (Client, OrderForm, Invoice, Service, Restaurant, Menu, Accessory, Flower) are implemented as independent object types with their own container. Their instances are identified through object identity (OID), without introducing an additional identifying attribute.

Generalization. The hierarchy of Service is maintained in the logical schema using type inheritance. Service is the parent type of Clothing, WeddingRegistryStore, FlowerService, and Catering. Clothing is further specialized into BridalClothing and GroomClothing. The hierarchy is kept directly in the object types, without introducing separate structures for the individual subtypes.

Multivalued attributes. Shoes and Stores remain simple multivalued attributes of Clothing, consistent with their conceptual treatment as attributes without independent identity.

Menu. At the conceptual level, Dish and Wine are separate entities connected to Menu through M:N relationships Contains and Offers. In the logical schema, these relationships are represented as collections inside Menu instead. Dish and Wine are consequently treated as components of a Menu rather than as independent objects. Menu, on the other hand, remains an independent object with its own identity, since it is considered an autonomous concept in the schema.

```

classDiagram
    class Client {
        +id: OID
    }
    class Invoice {
        +id: OID
    }
    class OrderForm {
        +wedding_date: DATE
        +filling_date: DATE
    }
    class Service {
        +description: VARCHAR(50)
        +min_cost: DECIMAL(8, 2)
        +max_cost: DECIMAL(8, 2)
        +lead_time: NUMBER
    }
    class RequestedServices {
    }
    class ProvidedServices {
        +cost: DECIMAL(8, 2)
    }
    class Restaurant {
        +name: VARCHAR(32)
        +address: VARCHAR(32)
    }
    class Catering {
    }
    class ProposedMenus {
    }
    class Menu {
        +id: OID
    }
    class Dish {
        +description: VARCHAR(50)
        +type: VARCHAR(16)
    }
    class Wine {
        +name: VARCHAR(32)
    }
    class Flower {
        +id: OID
    }
    class FlowerService {
        +arrangement_type: VARCHAR(32)
    }
    class ArrangementComposition {
    }
    class FlowerRegistryStore {
        +store_type: VARCHAR(16)
        +address: VARCHAR(32)
        +store_phone: VARCHAR(16)
    }
    class Clothing {
        +color: VARCHAR(16)
        +material: VARCHAR(16)
    }
    class BridalClothing {
    }
    class GroomClothing {
    }
    class Shoe {
        +name: VARCHAR(20)
    }
    class Store {
        +name: VARCHAR(20)
    }
    class Accessory {
        +material: VARCHAR(16)
        +type: VARCHAR(16)
        +cost: DECIMAL(8, 2)
    }
    class AccessoryMatched {
    }

    Client "1" -- "N" OrderForm
    Invoice "1" -- "N" OrderForm
    OrderForm "1" -- "N" RequestedServices
    RequestedServices "N" -- "N" Service
    Service "1" -- "N" ProvidedServices
    ProvidedServices "N" -- "1" Invoice
    Restaurant "1" -- "N" Catering
    Catering "1" -- "N" ProposedMenus
    ProposedMenus "1" -- "N" Menu
    Menu "1" -- "N" Dish
    Menu "1" -- "N" Wine
    Flower "1" -- "N" FlowerService
    FlowerService "1" -- "N" ArrangementComposition
    ArrangementComposition "1" -- "N" FlowerRegistryStore
    FlowerRegistryStore "1" -- "N" Clothing
    Clothing "1" -- "N" BridalClothing
    Clothing "1" -- "N" GroomClothing
    Shoe "N" -- "1" Clothing
    Store "N" -- "1" Clothing
    Accessory "1" -- "N" AccessoryMatched
    AccessoryMatched "N" -- "1" GroomClothing
  
```

13

5 Physical Design

5.1 Types and Tables definition

This paragraph describes the physical implementation of the system in Oracle. The types defined in the logical schema are implemented using Oracle object types and tables. It also shows how inheritance, embedded collections, and references are represented through the corresponding CREATE TYPE and CREATE TABLE statements.

Service

```
CREATE TYPE Service_ty AS OBJECT (  
    description VARCHAR(50),  
    min_cost     NUMBER(8,2),  
    max_cost     NUMBER(8,2),  
    lead_time    NUMBER  
) NOT INSTANTIABLE NOT FINAL;
```

Service_ty contains the attributes shared by all services. It is declared NOT FINAL so that it can be extended through inheritance: the four service categories are defined as subtypes of Service_ty using the UNDER keyword. The Service table can then store instances of Service_ty as well as instances of its subtypes.

Clothing

```
CREATE TYPE Shoe_ty AS OBJECT (  
    name VARCHAR(20)  
);  
/  
  
CREATE TYPE Store_ty AS OBJECT (  
    name VARCHAR(20)  
);  
/  
  
CREATE TYPE ShoeList_ty AS VARRAY(20) OF Shoe_ty;  
/  
  
CREATE TYPE StoreList_ty AS VARRAY(20) OF Store_ty;
```

Shoe and Store are defined as structured types without their own identity. They do not have separate CREATE TABLE statements and are stored directly within the outfit through the VARRAY collections ShoeList_ty and StoreList_ty. The VARRAYs are limited to 20 elements, which is an assumed maximum number of shoe models or fitting stores that can be associated with one outfit. This limit is needed to define the size of the VARRAY in Oracle.

```
CREATE TYPE Accessory_ty AS OBJECT (  
    material      VARCHAR(16),  
    accessory_type VARCHAR(16),  
    cost          NUMBER(8,2)  
);  
/  
  
CREATE TABLE Accessory OF Accessory_ty;  
/  
  
CREATE TYPE AccessoryMatched_ty AS OBJECT (  
    accessory_ref REF Accessory_ty
```

```
);
/

CREATE TYPE AccessoryMatched_nt AS TABLE OF AccessoryMatched_ty;
```

Accessory is kept as an independent typed table with its own identity, unlike Shoe and Store. The Matching relationship is represented by AccessoryMatched_ty, which contains only a reference to Accessory because the relationship has no attributes in the conceptual schema. AccessoryMatched_nt defines the nested table used to store the accessories matched to each groom outfit.

```
CREATE TYPE Clothing_ty UNDER Service_ty (
    color      VARCHAR(16),
    material   VARCHAR(16),
    shoes      ShoeList_ty,
    stores     StoreList_ty
) NOT INSTANTIABLE NOT FINAL;
/

CREATE TYPE BridalClothing_ty UNDER Clothing_ty ();
/

CREATE TYPE GroomClothing_ty UNDER Clothing_ty (
    accessories_matched AccessoryMatched_nt
);
```

Clothing_ty specializes Service_ty and adds the attributes and collections shared by the different types of outfits. It is declared NOT FINAL because it is further specialized into BridalClothing_ty and GroomClothing_ty. BridalClothing_ty does not add any attributes, so its type body is empty, while GroomClothing_ty adds the collection of matched accessories. No separate CREATE TABLE is defined for Clothing_ty or for either subtype. Their instances are stored in the Service table, which can also contain instances of the subtypes.

Wedding Registry Store

```
CREATE TYPE WeddingRegistryStore_ty UNDER Service_ty (
    store_type  VARCHAR(16),
    address     VARCHAR(32),
    store_phone VARCHAR(16)
);
```

Wedding registry stores do not require any auxiliary type or nested collection, since their information is represented entirely by scalar attributes. They directly specialize Service_ty and do not have a separate CREATE TABLE.

Flower Service

```
CREATE TYPE Flower_ty AS OBJECT (
    name        VARCHAR(32),
    shade       VARCHAR(16),
    start_month  NUMBER(2),
    end_month   NUMBER(2)
);
/

CREATE TABLE Flower OF Flower_ty(
    CONSTRAINT uq_flower UNIQUE (name, shade)
);
/
```

```

CREATE TYPE ArrangementComposition_ty AS OBJECT (
    flower_ref REF Flower_ty
);
/

CREATE TYPE ArrangementComposition_nt AS TABLE OF
    ArrangementComposition_ty;

```

Flower is kept as an independent typed table because a single flower can be used in multiple arrangements. The Composition relationship is represented by ArrangementComposition_ty, which contains only a reference to Flower because the relationship has no attributes in the conceptual schema. ArrangementComposition_nt defines the nested table used to store the flowers associated with each flower service.

```

CREATE TYPE FlowerService_ty UNDER Service_ty (
    arrangement_type VARCHAR(32),
    flowers            ArrangementComposition_nt
);

```

FlowerService_ty specializes Service_ty, adding its own scalar attribute and the collection of flowers associated with the service. Like every other category, no separate CREATE TABLE is defined for FlowerService_ty. Its instances are stored in the Service table.

Catering

```

CREATE TYPE Restaurant_ty AS OBJECT (
    name      VARCHAR(32),
    address   VARCHAR(32)
);
/

CREATE TABLE Restaurant OF Restaurant_ty(
    CONSTRAINT uq_restaurant UNIQUE (name, address)
);

```

Restaurant is implemented as an independent typed table and is referenced by Catering through a plain REF.

```

CREATE TYPE Dish_ty AS OBJECT (
    description VARCHAR(50),
    dish_type   VARCHAR(16)
);
/

CREATE TYPE Dish_nt AS TABLE OF Dish_ty;
/

CREATE TYPE Wine_ty AS OBJECT (
    name VARCHAR(32)
);
/

CREATE TYPE Wine_nt AS TABLE OF Wine_ty;
/

CREATE TYPE Menu_ty AS OBJECT (
    dishes Dish_nt,

```



```

        wines    Wine_nt
    );
/

CREATE TABLE Menu OF Menu_ty
    NESTED TABLE dishes STORE AS dishes_store
    NESTED TABLE wines  STORE AS wines_store;

```

Dish and Wine are embedded structured types without their own identity. Since they are accessed only through the Menu they belong to, they are implemented as nested tables within Menu_ty instead of separate typed tables. Each nested table column is defined with its own STORE AS clause. Menu_ty does not have any scalar attributes of its own.

```

CREATE TYPE ProposedMenus_ty AS OBJECT (
    menu_ref REF Menu_ty
);
/

CREATE TYPE ProposedMenus_nt AS TABLE OF ProposedMenus_ty;

```

ProposedMenus_ty represents the Propose relationship through a plain reference to Menu. Since Propose has no attributes in the conceptual schema, the type contains only the reference. ProposedMenus_nt defines the nested table used by a catering service to store its list of proposed menus.

```

CREATE TYPE Catering_ty UNDER Service_ty (
    restaurant_ref REF Restaurant_ty,
    proposed_menus ProposedMenus_nt
);

```

Catering_ty specializes Service_ty and contains a plain reference to its restaurant together with the collection of proposed menus. No separate CREATE TABLE is defined for Catering_ty, since its instances are stored in the Service table.

Finalizing the Service Table

```

CREATE TABLE Service OF Service_ty
    NESTED TABLE TREAT (SYS_NC_ROWINFO$ AS GroomClothing_ty).
        accessories_matched STORE AS accessories_matched_store
    NESTED TABLE TREAT (SYS_NC_ROWINFO$ AS FlowerService_ty).flowers STORE
        AS flowers_store
    NESTED TABLE TREAT (SYS_NC_ROWINFO$ AS Catering_ty).proposed_menus
        STORE AS proposed_menus_store;
/

ALTER TABLE accessories_matched_store
    ADD (SCOPE FOR (accessory_ref) IS Accessory);
/

ALTER TABLE flowers_store
    ADD (SCOPE FOR (flower_ref) IS Flower);
/

ALTER TABLE proposed_menus_store
    ADD (SCOPE FOR (menu_ref) IS Menu);

```

A single substitutable object table stores instances of Service_ty and its subtypes. The three collections defined by subtypes — accessories_matched in GroomClothing_ty, flowers in FlowerService_ty, and proposed_menus in Catering_ty — are implemented as nested tables rather than VARRAYs.

Each collection has its own storage table, where the REF stored by each element is represented as a column. This also allows the REFs to be declared as scoped and indexed if needed. Unlike a VARRAY, a nested table has no predefined limit on the number of elements.

Since these collections are declared in subtypes and not in `Service_ty`, they cannot be specified directly in the NESTED TABLE clauses. The row object must first be narrowed to the subtype that defines the corresponding attribute. This is done using `TREAT` on `SYS_NC_ROWINFO$`, which represents the complete row object of the object table. Rows whose actual type does not contain the corresponding attribute do not add any elements to that storage table. After the storage tables are created, their REF scopes are defined with `ALTER TABLE`, using the same approach as for `requested_services_store` and `provided_services_store`.

Client, Order Form and Invoice

```
CREATE TYPE Client_ty AS OBJECT (
    first_name VARCHAR(16),
    last_name  VARCHAR(16),
    phone      VARCHAR(16)
);
/

CREATE TABLE Client OF Client_ty(
    CONSTRAINT uq_client UNIQUE (first_name, last_name, phone)
);
```

Client is a plain, standalone entity, with no dependency on the Service hierarchy.

```
CREATE TYPE RequestedServices_ty AS OBJECT (
    service_ref REF Service_ty
);
/

CREATE TYPE RequestedServices_nt AS TABLE OF RequestedServices_ty;
/

CREATE TYPE OrderForm_ty AS OBJECT (
    wedding_date DATE,
    filing_date  DATE,
    client_ref   REF Client_ty,
    requested_services RequestedServices_nt
);
```

`RequestedServices_ty` represents the Request relationship through a plain reference to `Service`. Since Request has no attributes in the conceptual schema, the type contains only the reference. `OrderForm_ty` contains a plain REF to its Client and the `RequestedServices_nt` collection used to store its requested services.

```
CREATE TYPE ProvidedServices_ty AS OBJECT (
    cost          NUMBER(8,2),
    service_ref   REF Service_ty
);
/

CREATE TYPE ProvidedServices_nt AS TABLE OF ProvidedServices_ty;
/

CREATE TYPE Invoice_ty AS OBJECT (
    issue_date    DATE,
```

```

    wedding_date      DATE,
    payment_method     VARCHAR(32),
    client_ref         REF Client_ty,
    provided_services  ProvidedServices_nt
);

```

ProvidedServices_ty is structured in the same way, but also includes the FinalCost attribute because Provision has an attribute in the conceptual schema. Invoice_ty is similar to OrderForm_ty, with a plain reference to Client and a nested collection of provided services. It also stores the wedding_date, which identifies the specific wedding when a client has organized more than one.

```

CREATE TABLE OrderForm OF OrderForm_ty (
    SCOPE FOR (client_ref) IS Client
)
NESTED TABLE requested_services STORE AS requested_services_store;
/

ALTER TABLE requested_services_store
    ADD (SCOPE FOR (service_ref) IS Service);
/

CREATE TABLE Invoice OF Invoice_ty(
    SCOPE FOR (client_ref) IS Client
)
NESTED TABLE provided_services STORE AS provided_services_store;
/

ALTER TABLE provided_services_store
    ADD (SCOPE FOR (service_ref) IS Service);

```

OrderForm_ty and Invoice_ty are each materialized in a separate table. Since their nested collections are declared directly in the corresponding types, the STORE AS clauses can refer to the columns directly, without requiring the TREAT used for the collections defined in the Service subtypes.

Scopes and integrity constraints

All REFs stored in nested-table storage tables, as well as client_ref in OrderForm and Invoice, are declared as scoped REFs. This includes service_ref in requested_services_store and provided_services_store, and accessory_ref, flower_ref, and menu_ref in accessories_matched_store, flowers_store, and proposed_menus_store.

The scope is defined at table level, so it is specified in the CREATE TABLE or ALTER TABLE statement of the table containing the REF column. The REF type only determines the type of the referenced object, while the scope further restricts the reference to a specific table. For example, service_ref can only refer to an object stored in the Service table. Scoped REFs also require less storage than unscoped REFs and can be indexed, as discussed in Section 5.4.

UNIQUE constraints are used only where a suitable natural key can be identified. For Client, the combination (first_name, last_name, phone) prevents exact duplicates while still allowing different clients with the same name. For Flower, (name, shade) is used because the same species in different shades represents different rows. Restaurant uses (name, address) as its unique combination.

5.2 Triggers

Not every business rule needs a trigger. BR2, which concerns the order of FilingDate, WeddingDate, and IssueDate, and BR4, which compares min_cost and max_cost, involve only attributes of the same row. BR6 restricts dish_type to a fixed set of values, while BR7 checks the two attributes defining the flower's availability period. These four rules can therefore be enforced with CHECK constraints

on OrderForm, Invoice, Service, Flower, and the dishes_store nested-table storage, without requiring procedural logic. BR1 is different: it is a modeling assumption about the relationship between invoices and order forms rather than a condition that can be violated by a single DML operation. It therefore has no corresponding trigger or constraint.

The remaining rules require access to data outside the row being checked. BR8 compares an Invoice with the corresponding OrderForm, while BR3 and BR5 require checking every element of a nested collection through a REF, respectively for the lead time and cost of requested or provided services. Since Oracle CHECK constraints cannot handle these cases, the three rules are implemented using triggers.

BR2, BR4, BR6, BR7 - Check constraints

```
ALTER TABLE OrderForm
  ADD CONSTRAINT chk_orderform_dates
  CHECK (filing_date < wedding_date);
/

ALTER TABLE Invoice
  ADD CONSTRAINT chk_invoice_dates
  CHECK (issue_date >= wedding_date);
/

ALTER TABLE Service
  ADD CONSTRAINT chk_service_costs
  CHECK (min_cost <= max_cost);
/

ALTER TABLE Flower
  ADD CONSTRAINT chk_flower_period
  CHECK (start_month BETWEEN 1 AND 12 AND end_month BETWEEN 1 AND 12);
/

ALTER TABLE dishes_store
  ADD CONSTRAINT chk_dish_type
  CHECK (dish_type IN ('appetizer', 'first course', 'second course', '
    dessert'));
```

chk_orderform_dates and chk_invoice_dates enforce BR2 on OrderForm and Invoice, respectively, while chk_service_costs enforces BR4. All three constraints compare two scalar attributes within the same row. chk_flower_period enforces BR7 by checking that the start and end months are valid. For BR6, chk_dish_type is defined on dishes_store because dishes is a nested-table column of Menu, so the constraint must be applied to the storage table containing the collection elements.

BR3 - Lead time for requested services

Given an order form, every service in its requested_services collection must leave enough time between filing_date and wedding_date to honor that service's lead_time.

```
CREATE OR REPLACE TRIGGER trg_orderform_leadtime
BEFORE INSERT OR UPDATE ON OrderForm
FOR EACH ROW
DECLARE
  v_available NUMBER;
  v_service   Service_ty;
BEGIN
  v_available := :NEW.wedding_date - :NEW.filing_date;

  IF :NEW.requested_services IS NOT NULL THEN
    FOR i IN 1 .. :NEW.requested_services.COUNT LOOP
```

```

        SELECT Deref(:NEW.requested_services(i).service_ref)
        INTO v_service
        FROM DUAL;

        IF v_available < v_service.lead_time THEN
            RAISE_APPLICATION_ERROR(-20002, 'The interval between
            filing date and wedding date is shorter than the lead
            time of one of the requested services.');
```

:NEW.requested_services is accessed directly as a PL/SQL collection, so no TABLE() operator is required. Each element's service_ref is then resolved with Deref to access the referenced Service_ty instance and read its lead_time.

Deref is a SQL operator rather than a PL/SQL function, so it cannot be assigned directly to a variable such as *v_service := Deref(...)*. The expression is instead evaluated through a SELECT ... INTO ... FROM DUAL. Here, DUAL is used only to evaluate the expression and store its result in the variable; no actual table data is being queried.

BR5 - Final cost within min/max range

Given an invoice, every entry in its provided_services collection must report a cost between the min_cost and max_cost of the referenced service.

```

CREATE OR REPLACE TRIGGER trg_invoice_finalcost
BEFORE INSERT OR UPDATE ON Invoice
FOR EACH ROW
DECLARE
    v_service Service_ty;
BEGIN
    IF :NEW.provided_services IS NOT NULL THEN
        FOR i IN 1 .. :NEW.provided_services.COUNT LOOP
            SELECT Deref(:NEW.provided_services(i).service_ref)
            INTO v_service
            FROM DUAL;

            IF :NEW.provided_services(i).cost < v_service.min_cost OR :NEW
                .provided_services(i).cost > v_service.max_cost THEN
                RAISE_APPLICATION_ERROR(-20003, 'The final cost of a
                provided service must fall between its minimum and
                maximum cost.');
```

The trigger follows the same pattern as trg_orderform_leadtime: it iterates over the collection and dereferences each element, but this time it compares the cost stored in the association object with the min_cost and max_cost values of the referenced Service_ty instance.

BR8 - Invoice referring to an existing wedding

Since an invoice stores the date of the wedding it refers to, the invoice part of BR2 can be checked directly with chk_invoice_dates. BR8 requires a different check: the wedding date recorded in the invoice must correspond to an actual wedding organized by that client, meaning that at least one of the client's order forms must have the same date. This requires a lookup on OrderForm, so it is enforced through a trigger.

```

CREATE OR REPLACE TRIGGER trg_invoice_wedding_exists
```

```

BEFORE INSERT OR UPDATE ON Invoice
FOR EACH ROW
DECLARE
    v_count NUMBER;
BEGIN
    SELECT COUNT(*)
    INTO v_count
    FROM OrderForm o
    WHERE o.client_ref = :NEW.client_ref
        AND o.wedding_date = :NEW.wedding_date;

    IF v_count = 0 THEN
        RAISE_APPLICATION_ERROR(-20001, 'The wedding date of the invoice
            does not match any order form of the client.');
```

```

    END IF;
END;
```

5.3 Procedures and Functions

Several procedures were implemented to support the four operations, with each operation defined as a separate stored procedure. Unlike in a purely relational schema, the independent entities in this project do not use numeric keys. Their instances are identified by system-generated OIDs, so procedures that operate on a specific instance receive a REF to that object as a parameter and use Deref to navigate the schema.

Procedure 1: Insert a new client

This procedure inserts a new client into the database. It takes the client's first name, last name, and phone number as parameters, creates a Client_ty instance with these values, and inserts it into the Client table.

```

CREATE OR REPLACE PROCEDURE insert_new_client (
    p_first_name IN VARCHAR,
    p_last_name  IN VARCHAR,
    p_phone      IN VARCHAR
) IS
BEGIN
    INSERT INTO Client
    VALUES (Client_ty(p_first_name, p_last_name, p_phone));
END insert_new_client;
```

Procedure 2: Insert a new order form

This procedure inserts a new order form for a given client together with the requested services. It takes a REF to the client, the wedding and filing dates, and a list of REFs to the requested services. The list is passed as an instance of the schema-level collection type ServiceRef_nt and each REF is converted into a RequestedServices_ty object before being added to the nested collection of OrderForm_ty.

```

CREATE OR REPLACE TYPE ServiceRef_nt AS TABLE OF REF Service_ty;
/

CREATE OR REPLACE PROCEDURE insert_order_form (
    p_client_ref  IN REF Client_ty,
    p_wedding_date IN DATE,
    p_filing_date IN DATE,
    p_service_refs IN ServiceRef_nt
) IS
    v_requested_services RequestedServices_nt := RequestedServices_nt();
BEGIN
```

```

FOR i IN 1 .. p_service_refs.COUNT LOOP
    v_requested_services.EXTEND;
    v_requested_services(i) := RequestedServices_ty(p_service_refs(i))
;
END LOOP;

INSERT INTO OrderForm
VALUES (
    OrderForm_ty(
        p_wedding_date,
        p_filing_date,
        p_client_ref,
        v_requested_services
    )
);
END insert_order_form;

```

The insertion is also checked by `trg_orderform_leadtime`. If any requested service does not satisfy the required lead time, the trigger raises an exception before the row is inserted.

Procedure 3: Print catering details

This procedure prints the restaurant and the complete content of each proposed menu, including dishes and wines, for a given catering service. It takes a REF to `Service_ty` as its only parameter and uses `TREAT` to narrow it to `Catering_ty`, since the REF may point to any service stored in the substitutable `Service` table.

```

CREATE OR REPLACE PROCEDURE print_catering_details (
    p_service_ref IN REF Service_ty
) IS
    v_catering    Catering_ty;
    v_restaurant  Restaurant_ty;
    v_menu        Menu_ty;
BEGIN
    SELECT TREAT(DEREF(p_service_ref) AS Catering_ty)
    INTO v_catering
    FROM DUAL;

    IF v_catering IS NULL THEN
        RAISE_APPLICATION_ERROR(-20010, 'The given service is not a
            catering service. ');
    END IF;

    SELECT Deref(v_catering.restaurant_ref)
    INTO v_restaurant
    FROM DUAL;

    DBMS_OUTPUT.PUT_LINE('Restaurant: ' || v_restaurant.name || ', ' ||
        v_restaurant.address);

    IF v_catering.proposed_menus IS NOT NULL THEN
        FOR i IN 1 .. v_catering.proposed_menus.COUNT LOOP
            SELECT Deref(v_catering.proposed_menus(i).menu_ref)
            INTO v_menu
            FROM DUAL;

            DBMS_OUTPUT.PUT_LINE('--- Menu ---');

            IF v_menu.dishes IS NOT NULL THEN
                FOR d IN 1 .. v_menu.dishes.COUNT LOOP
                    DBMS_OUTPUT.PUT_LINE(' Dish: ' || v_menu.dishes(d).
                        description

```

```

|| ' (' || v_menu.dishes(d).dish_type ||
|| ')');

END LOOP;
END IF;

IF v_menu.wines IS NOT NULL THEN
  FOR w IN 1 .. v_menu.wines.COUNT LOOP
    DBMS_OUTPUT.PUT_LINE(' Wine: ' || v_menu.wines(w).
      name);
  END LOOP;
END IF;
END LOOP;
END IF;
END print_catering_details;

```

If the referenced object is not a Catering_ty instance, TREAT returns NULL instead of raising an exception.

Procedure 4: Print requested services of a client

This procedure prints all services requested by a given client across their order forms, including the description, minimum and maximum cost, and category of each service. It takes a REF to Client_ty as its only parameter and retrieves the data with a single query. The requested_services collection of each order form is unnested with TABLE and joined with the Service table using the referenced object. The category is determined directly in the SELECT list with IS OF.

```

CREATE OR REPLACE PROCEDURE print_client_services (
  p_client_ref IN REF Client_ty
) IS
BEGIN
  FOR rec IN (
    SELECT s.description, s.min_cost, s.max_cost,
      CASE
        WHEN VALUE (s) IS OF (Clothing_ty) THEN 'Clothing'
        WHEN VALUE (s) IS OF (WeddingRegistryStore_ty) THEN '
          Wedding Registry Store'
        WHEN VALUE (s) IS OF (FlowerService_ty) THEN 'Flower
          Service'
        WHEN VALUE (s) IS OF (Catering_ty) THEN 'Catering'
        ELSE 'Unknown'
      END AS category
    FROM OrderForm o, TABLE (o.requested_services) r, Service s
    WHERE o.client_ref = p_client_ref AND REF(s) = r.service_ref
  ) LOOP
    DBMS_OUTPUT . PUT_LINE (
      'Service : ' || rec.description
      || ', Min Cost : ' || rec.min_cost
      || ', Max Cost : ' || rec.max_cost
      || ', Category : ' || rec.category
    );
  END LOOP ;
END print_client_services ;

```

5.4 Query Optimization

The indexes were chosen by looking at how each of the four operations accesses the data.

Operation 1, which inserts a client, does not require any index analysis because it performs only an INSERT and no SELECT. The same applies to Operation 2, implemented by print_catering_details: the procedure receives a REF to Service_ty and accesses the restaurant, menus, dishes, and wines by following the existing references, without filtering data through a WHERE clause. Operation 3, which

inserts an order form, also performs no lookup. It inserts a fully constructed OrderForm_ty object using REFs already provided by the caller.

Operation 4, implemented by print_client_services, is different because its query filters data on a non-key column. The procedure executes a single query that unnests the requested services of the client's order forms and joins them with the Service table:

```
SELECT s.description, s.min_cost, s.max_cost, ...
FROM OrderForm o, TABLE (o.requested_services) r, Service s
WHERE o.client_ref = p_client_ref AND REF(s) = r.service_ref;
```

The two join operations can use structures that Oracle creates automatically: an index on the NESTED_TABLE_ID column of requested_services_store, which links each requested service to its order form, and an index on the object identifier of the Service table. The only step without a supporting structure is the selection of the client's order forms.

The condition on OrderForm is a simple equality lookup. print_client_services only compares client_ref with a specific client; it does not use range conditions or sort by this attribute. A secondary B-tree index on client_ref therefore matches the access pattern. Instead of scanning all 6,000 OrderForm rows, Oracle can use the index to find the matching entries and retrieve only those rows.

A hash cluster was also considered because it could group the order forms of the same client together. Since a client has on average $6,000/1,500 = 4$ order forms, this could reduce the retrieval to roughly one block access. However, Oracle only allows scalar data types as cluster keys, while client_ref is a REF. It therefore cannot be used as a cluster key.

A secondary index has a less restrictive requirement. Oracle does not allow indexes on unscoped REF columns because an unscoped REF may point to different tables containing the referenced type. Once the REF is scoped to a specific table, however, it can be indexed. Since client_ref is a scoped REF, a B-tree index can be created on it, making this the only secondary structure used for this access path:

```
CREATE INDEX idx_orderform_client_ref ON OrderForm(client_ref);
```

To evaluate the effect of the index, the query used by print_client_services was analyzed with Oracle SQL Developer's Explain Plan both before and after creating the index. The CASE expression used to determine the service category was omitted. In both runs, the data volume, optimizer statistics, and client used for filtering were kept unchanged. The parameter p_client_ref was replaced with a subquery that retrieves the REF of a specific client:

```
SELECT s.description, s.min_cost, s.max_cost
FROM OrderForm o, TABLE (o.requested_services) r, Service s
WHERE o.client_ref = (
    SELECT REF(c)
    FROM Client c
    WHERE c.first_name = 'FirstName1' AND c.last_name = 'LastName1'
)
AND REF(s) = r.service_ref;
```

Figure 4 shows the plan without the index. OrderForm is accessed through a TABLE ACCESS FULL with a cost of 19, giving a total statement cost of 57. After creating the index, as shown in Figure 5, OrderForm is accessed through an INDEX RANGE SCAN on idx_orderform_client_ref with a cost of 2, while the total cost falls to 40. Since the rest of the plan remains unchanged, the reduction of 17, or about 30%, is entirely due to the different access path for OrderForm.

SQL | 0,003 secondi

OPERATION	OBJECT_NAME	OPTIONS	CARDINALITY	COST
SELECT STATEMENT				180
HASH JOIN				57
Access Predicates				180
S.SYS_NC_OID\$=R.SERVICE_REF				54
HASH JOIN				180
Access Predicates				49
R.NESTED_TABLE_ID=O.SYS_NC0000600007\$				
TABLE ACCESS	ORDERFORM	FULL		60
Filter Predicates				19
O.CLIENT_REF= (SELECT REF(C) FROM CLIENT C WHERE C.LAST_NAME='LastName1' AND C.FIRST_NAME='FirstName1')				
TABLE ACCESS	CLIENT	BY INDEX ROWID BATCHED	1	3
INDEX	UQ_CLIENT	RANGE SCAN	1	2
Access Predicates				
AND				
C.FIRST_NAME='FirstName1'				
C.LAST_NAME='LastName1'				
TABLE ACCESS	REQUESTED_SERVICES_STORE	FULL		18000
TABLE ACCESS	SERVICE	FULL		500

Figure 4: Explain Plan without index on client_ref

SQL | 0,005 secondi

OPERATION	OBJECT_NAME	OPTIONS	CARDINALITY	COST
SELECT STATEMENT				180
HASH JOIN				40
Access Predicates				180
S.SYS_NC_OID\$=R.SERVICE_REF				37
HASH JOIN				180
Access Predicates				32
R.NESTED_TABLE_ID=O.SYS_NC0000600007\$				
TABLE ACCESS	ORDERFORM	BY INDEX ROWID BATCHED	60	2
INDEX	IDX_ORDERFORM_CLIENT_REF	RANGE SCAN	24	1
Access Predicates				
O.CLIENT_REF= (SELECT REF(C) FROM CLIENT C WHERE C.LAST_NAME='LastName1' AND C.FIRST_NAME='FirstName1')				
TABLE ACCESS	CLIENT	BY INDEX ROWID BATCHED	1	3
INDEX	UQ_CLIENT	RANGE SCAN	1	2
Access Predicates				
AND				
C.FIRST_NAME='FirstName1'				
C.LAST_NAME='LastName1'				
TABLE ACCESS	REQUESTED_SERVICES_STORE	FULL		18000
TABLE ACCESS	SERVICE	FULL		500

Figure 5: Explain Plan with index on client_ref

6 Data Warehouse

6.1 Facts

The fact tables are based on recurring business events that are useful to analyze over time and across different dimensions, rather than individual instances as in the four operational queries. The requirements identify two such events: a client requesting a service, represented by the Request relationship, and a service being provided and invoiced, represented by the Provision relationship. These events are distinct because a requested service is not necessarily provided, and Request also contains information that is not present in Provision, such as the lead time. For this reason, two fact tables are used.

- **Fact 1 - Service Request.** This fact can be used, for example, to compare the lead time of different service categories or to analyze how often each category is requested over time.
- **Fact 2 - Service Provision.** This fact can be used to analyze revenue by service category and period, or to compare the final cost with the corresponding catalog price range.

The two facts share the same general context: a service associated with a client and a wedding at a given point in time. They can therefore use common dimensions such as Date, WeddingDate, Client, and Service. This leads to a constellation schema, where multiple fact tables share the same dimension tables instead of maintaining separate copies of them.

6.2 Measures

Fact 1 - Service Request

- **RequestCount:** the number of times a service is requested; enables analysis of which services are most popular and how that varies over time.
- **BookingAdvanceDays:** the number of days between filing date and wedding date; enables analysis of how far in advance clients book services.

Fact 2 - Service Provision

- **ProvisionCount:** the number of times a service is provided; enables analysis of which services are actually delivered and invoiced.
- **FinalCost:** the final cost charged for the service; enables analysis of revenue and pricing trends.
- **MarginFromMin:** the difference between FinalCost and the service's MinCost; enables analysis of how much above the minimum price services are being sold for.
- **MarginToMax:** the difference between the service's MaxCost and FinalCost; enables analysis of how much below the maximum price services are being sold for.

FinalCost, MarginFromMin, and MarginToMax are fully additive measures, so they can be summed across any dimension or combination of dimensions, such as date, client, or service category. RequestCount and ProvisionCount are counting measures that have a value of 1 and become useful when aggregated. BookingAdvanceDays is non-additive: summing it does not have a meaningful interpretation, while its average, minimum, and maximum can be used to analyze booking behavior.

The two margin measures could be calculated from FinalCost and the MinCost and MaxCost values stored in the Service dimension. They are instead stored in the fact and calculated during ETL because they refer to the price range that was in effect when the service was provided.

6.3 Dimensions

- **Date:** In Fact 1, it represents the FilingDate of the request and is referenced through idFilingDate; in Fact 2, it represents the IssueDate of the invoice and is referenced through idIssueDate. The dimension has the standard Day → Month → Quarter → Year hierarchy and an IsHoliday flag, computed during ETL.

- **WeddingDate:** shared by both facts and represents the wedding date associated with the requested or provided service. It follows the same Day → Month → Quarter → Year hierarchy and includes an IsHoliday flag computed during ETL. In Fact 2, the value comes directly from the invoice, which stores the date of the wedding it refers to.
- **Client:** shared by both facts, with one row for each client and the attributes FirstName, LastName, and Phone. The requirements do not provide any geographical information that could be used for further aggregation, so the dimension has no hierarchy beyond the client level.
- **Service:** shared by both facts, with one row for each service. It includes Description, MinCost, MaxCost, LeadTime, and Category, with Service → Category as its two-level hierarchy. The four categories correspond to the service generalization in the ER schema: Clothing, Wedding Registry Store, Flower Service, and Catering.
- **PaymentMethod:** used only by Fact 2, since requests do not have a payment method. It contains one row for each distinct payment method value.

6.4 The Constellation Schema

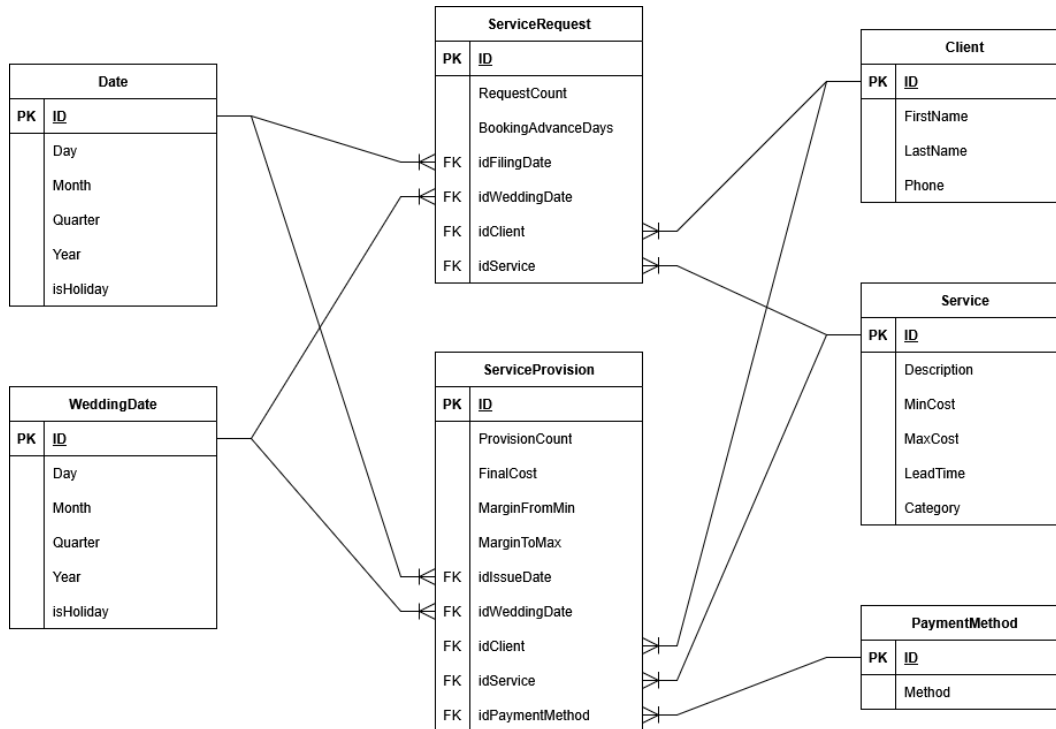


Figure 6: Constellation Schema

Figure 6 shows the resulting data warehouse schema. The two fact tables, **ServiceRequest** and **ServiceProvision**, are at the center of the diagram, together with the dimensions used by each one. Since **Date**, **WeddingDate**, **Client**, and **Service** are shared by both facts, the schema forms a fact constellation rather than two separate star schemas.

7 Web Application

To show the database at work beyond the SQL Developer environment, a small web application was developed on top of the Oracle schema described in the previous sections. The application provides a browser interface for the four workload operations defined in Section 4.

7.1 Architecture

The application uses a classic three-tier architecture, shown in Figure 7. The presentation tier is the user's browser, which displays the HTML pages returned by the application. The application tier is a Python program built with Flask, a lightweight web framework. It handles HTTP requests, executes the SQL or PL/SQL statements required by each operation, and generates the corresponding HTML pages. The data tier is the Oracle database, which contains the schema described in Section 5, together with its procedures, triggers, and constraints.

Flask communicates with Oracle through python-oracledb, Oracle's official Python driver. The driver runs in thin mode, so it connects directly to the database over the network without requiring the Oracle Client libraries on the application machine. Thin mode also supports Oracle object types, allowing the application to receive user-defined types as structured Python objects, including nested tables such as those used for the contents of a menu.

Each workload operation is exposed through a separate page. The root page acts as a dashboard, providing links to the four operations and showing the current Oracle connection status. When one of the pages is requested, the application opens a database connection, executes the statements needed for the selected operation, and closes the connection before returning the generated page to the browser.

The connection parameters, including host, port, service name, user, and password, are read from environment variables. This allows the same application code to be used with different database instances without changing the source code. The application listens only on the local interface (127.0.0.1), since it is intended to run as a local client of the database.



Figure 7: Web Application Architecture

7.2 Connection Management

Opening a connection to Oracle is relatively expensive because it involves a network round trip, user authentication, and the creation of a new server session. Since each workload operation executes only a few short statements, creating a new connection for every request could take more time than the operation itself. The application therefore uses a connection pool instead of opening connections directly. The pool keeps a set of database sessions open and reuses them across requests:

```
_pool = oracledb.create_pool(  
    user=os.getenv("ORACLE_USER"),  
    password=os.getenv("ORACLE_PASSWORD"),  
    dsn=database_dsn(),  
    min=1,  
    max=5,  
    increment=1,  
)
```

The pool keeps at least one session open and can grow up to a maximum of five sessions, adding them as needed. This limit controls the number of sessions that the application can use at the same time. If all five sessions are already in use, a new request waits for one to become available instead of opening another session.

Every page obtains its connection through the same helper function. The function acquires a session from the pool at the start of the operation and returns it when the operation is finished, including when the operation raises an error:

```
@contextmanager
def get_connection():
    connection = get_pool().acquire()
    try:
        yield connection
    finally:
        connection.close()
```

In this context, calling `close()` on a pooled connection does not terminate the server-side session. It simply returns the connection to the pool so that it can be reused by another request.

The pool is created lazily, when the first request needs a connection, rather than when the application starts. This also allows the application to start when the database is temporarily unreachable. If pool creation fails, the error is handled by the page that requested the connection, and the application tries to create the pool again on the next request. Once the database becomes available, the application can therefore resume working without being restarted. When the application terminates, the pool is closed and all its sessions are released.

The dashboard also uses a pooled connection to show the database status. It runs a minimal query that reads the user and service name from the session context:

```
SELECT USER, SYS_CONTEXT('USERENV', 'SERVICE_NAME') FROM DUAL
```

If the query succeeds, the dashboard displays both values; otherwise, it reports that Oracle is unavailable together with the error returned by the driver.

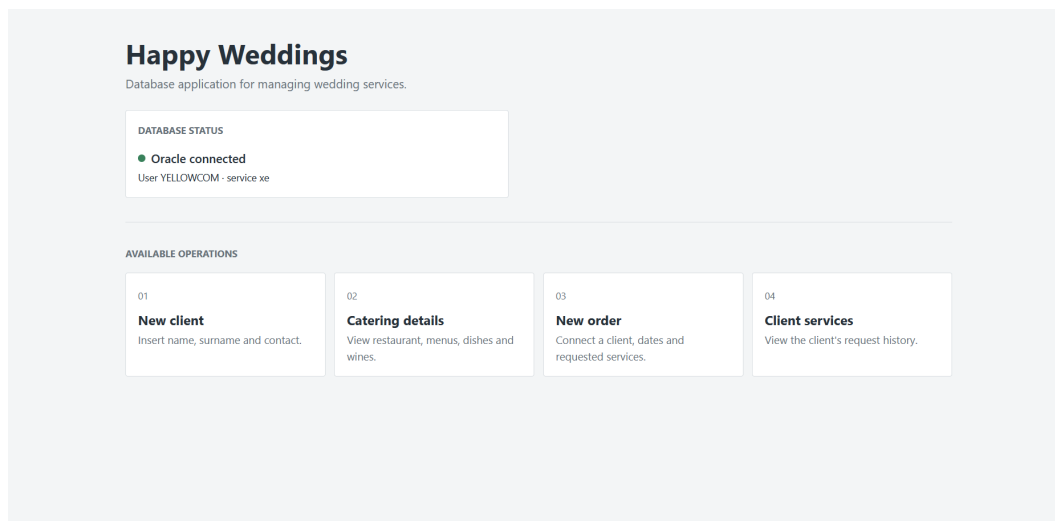


Figure 8: Web Application - Dashboard

7.3 Accessing the Object-Relational Schema

This section explains how the application interacts with the object-relational features of the schema, in particular object identity and REFs, the substitutable Service hierarchy, and nested tables.

Identifying objects. The procedures of the schema identify objects through REFs rather than artificial keys. REFs cannot be placed directly in an HTML page and sent back by the browser as form values, so the application encodes the REF of the corresponding row as a hexadecimal string using the built-in function REFTOHEX, and decodes it back into a REF with HEXTOREF when required by a procedure or query.

```
SELECT REFTOHEX(REF(c)) FROM Client c
...
WHERE REF(c) = HEXTOREF(:client_ref)
```

Operation 1: inserting a client. The page directly calls the insert_new_client procedure described in Section 5.3, passing the three values entered in the form as parameters. The Client_ty instance is created inside the procedure.

OPERATION 01

New client

Add a person to the Happy Weddings registry.

First name

Last name

Phone

Figure 9: Web Application - Insert Client

Operation 2: printing catering details. The print_catering_details procedure described in Section 5.3 writes its output using DBMS_OUTPUT. This output is displayed by development tools such as SQL Developer, but it cannot be retrieved by the application as a result set. The application therefore performs the same navigation using SQL queries. The page first obtains the list of catering services by filtering the substitutable Service table according to the dynamic type of its rows:

```
SELECT REFTOHEX(REF(s)), description FROM Service s
WHERE VALUE(s) IS OF (Catering_ty)
ORDER BY description
```

Once a catering service has been selected, its restaurant is reached by narrowing the row to Catering_ty with TREAT and dereferencing restaurant_ref:

```
SELECT Deref(TREAT(VALUE(s) AS Catering_ty).restaurant_ref).name, Deref(
    TREAT(VALUE(s) AS Catering_ty).restaurant_ref).address
FROM Service s
WHERE REF(s) = HEXTOREF(:service_ref)
```

Its menus are obtained by unnesting the proposed_menus collection of the subtype and joining each element with the Menu table on the referenced object:

```
SELECT m.dishes, m.wines
FROM Service s, TABLE(TREAT(VALUE(s) AS Catering_ty).proposed_menus)
    proposed, Menu m
WHERE REF(s) = HEXTOREF(:service_ref)
```

```
AND      REF(m) = proposed.menu_ref
```

The last query returns the dishes and wines nested tables of each menu as two columns. Instead of unnesting these collections further in SQL, the application receives them as values of type Dish_nt and Wine_nt. Python-oracledb maps each value to a collection object, whose elements are converted into lists of Dish_ty and Wine_ty instances. Their attributes can then be accessed by name:

```
for dish in dishes.asList():
    dish.DESCRPTION, dish.DISH_TYPE
```

In this way, a menu is transferred from the database to the application with its structure preserved, exactly as it is stored.

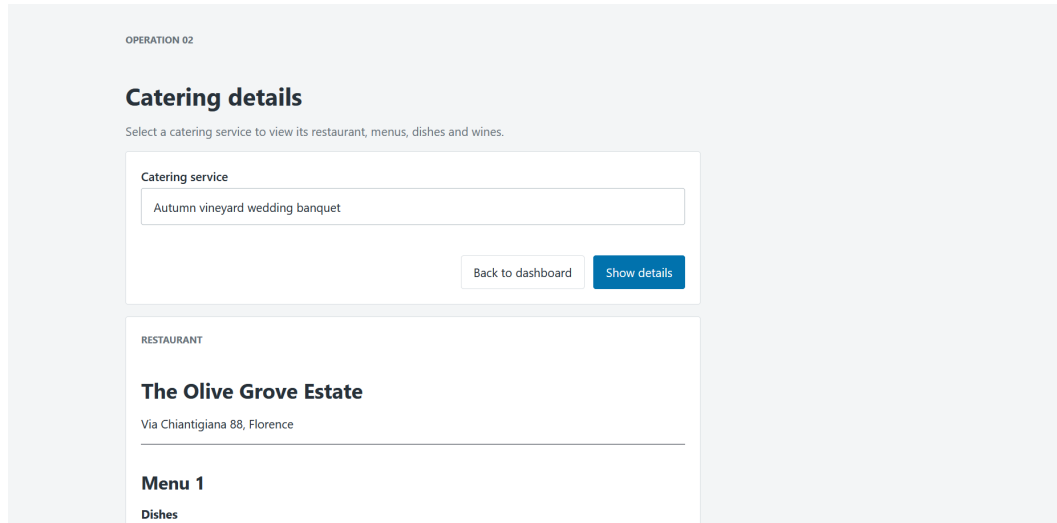


Figure 10: Web Application - Catering Details - Restaurant

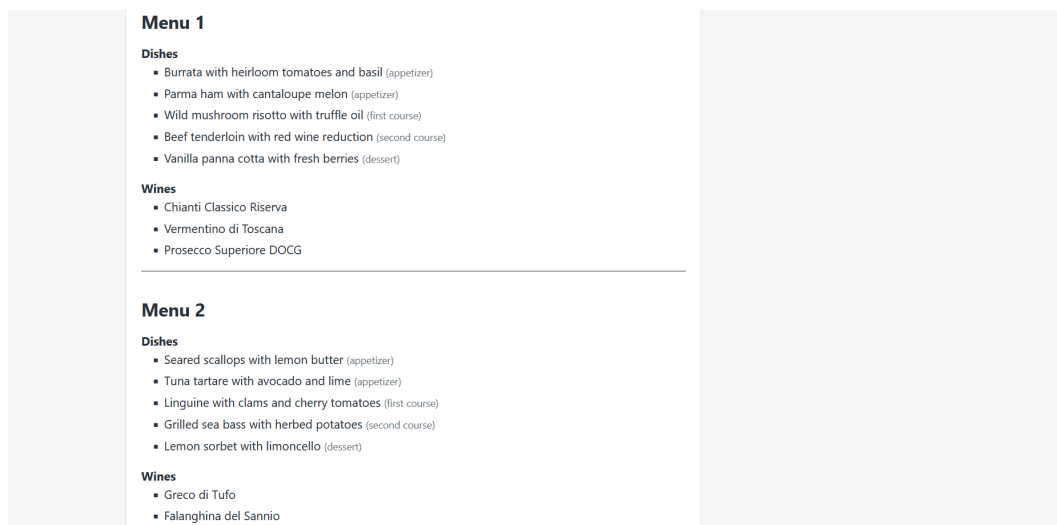


Figure 11: Web Application - Catering Details - Menus

Operation 3: inserting an order form. The insert_order_form procedure expects a REF to the client and a ServiceRef_nt collection containing the REFs of the requested services. Since the application only receives the hexadecimal encoding of these REFs, it uses an anonymous PL/SQL block that decodes each value with HEXTOREF as it is assigned to a local variable, and collects the

requested services into a ServiceRef_nt before calling the procedure. For an order form requesting two services, the block is:

```
DECLARE
    v_services ServiceRef_nt := ServiceRef_nt();
    v_client    REF Client_ty := HEXTOREF(:client_ref);
BEGIN
    v_services.EXTEND;
    v_services(1) := HEXTOREF(:service_0);
    v_services.EXTEND;
    v_services(2) := HEXTOREF(:service_1);
    insert_order_form(v_client, TO_DATE(:wedding_date, 'YYYY-MM-DD'),
        TO_DATE(:filing_date, 'YYYY-MM-DD'), v_services);
END;
```

The application generates one decoding step for each selected service, but it never inserts the user's input into the text of the block: dates and encoded REFs are always passed as bind variables.

Figure 12: Web Application - Insert Order Form

Operation 4: listing the services of a client. For the same reason of operation 2, the application does not call `print_client_services` directly. Instead, it executes the query described in Section 5.3. The client is identified by decoding the value received from the browser back into a REF with `HEXTOREF`, and the query also returns the wedding date associated with each order form. This keeps the services requested for different weddings of the same client separate:

```
SELECT o.wedding_date, s.description, s.min_cost, s.max_cost,
    CASE
        WHEN VALUE(s) IS OF (Clothing_ty) THEN 'Clothing'
        WHEN VALUE(s) IS OF (WeddingRegistryStore_ty) THEN 'Wedding
            registry store'
        WHEN VALUE(s) IS OF (FlowerService_ty) THEN 'Flower service'
        WHEN VALUE(s) IS OF (Catering_ty) THEN 'Catering'
        ELSE 'Service'
    END AS category
FROM OrderForm o, TABLE(o.requested_services) requested, Service s
WHERE o.client_ref = HEXTOREF(:client_ref) AND REF(s) = requested.
    service_ref
ORDER BY o.wedding_date, s.description
```

OPERATION 04

Client services

Select a client to view all services requested in their orders.

Clients

Andrea Rossi
123456789

FirstName1 LastName1
3488099269

FirstName10 LastName10
3016800049

FirstName100 LastName100
3666039469

FirstName1000 LastName1000
3544962835

FirstName1001 LastName1001

Figure 13: Web Application - Client Services

OPERATION 04

Requested services

Request history for the selected client.

Service	Category	Wedding date	Minimum cost	Maximum cost
Bridal outfit 1	Clothing	23/05/2028	1289.84	3716.62
Bridal outfit 19	Clothing	23/05/2028	673.42	1554.99
Catering package 8	Catering	23/05/2028	2034.73	4514.21
Flower service 136	Flower service	23/05/2028	456.17	1316.9
Groom outfit 32	Clothing	23/05/2028	343.51	2566.76
Registry store 2	Wedding registry store	23/05/2028	231.73	525.82

Back to client list

Back to dashboard

Figure 14: Web Application - Requested Services of a Client

7.4 Transactions and Integrity

Every operation that modifies data is executed as a single transaction using a connection obtained from the pool. The `insert_new_client` and `insert_order_form` procedures do not issue a `COMMIT`, so the transaction is committed by the application only after the procedure call succeeds. If an error occurs, the commit is skipped and the uncommitted changes are rolled back when the connection is returned to the pool.

This is especially important for Operation 3. The anonymous block described in Section 7.3 first converts the client and the selected services into REFs, then inserts the order form together with its nested collection of requested services. All of these operations are part of the same transaction, so a failure prevents the order form and its requested services from being stored partially.

The application does not duplicate the database business rules. The forms only perform basic validation for usability, such as checking required fields, maximum lengths matching the corresponding column sizes, and whether at least one service has been selected for an order form. The actual rules are enforced by the database through the constraints and triggers described in Section 5.

If a statement violates one of these rules, Oracle raises an error and `python-oracledb` reports it to the

application as an exception. The application catches the exception without committing the transaction and displays the error message on the same page. The user can then correct the input and submit the form again. For rules enforced by triggers, the displayed message comes from the corresponding RAISE_APPLICATION_ERROR call. For example, if an order form is submitted with a filing date that does not provide enough lead time for one of its requested services, the application displays the error raised by trg_orderform_leadtime.

Could not register order: ORA-20002: The interval between filing date and wedding date is shorter than the lead time of one of the requested services.
ORA-06512: at "YELLOWCOM.TRG_ORDERFORM_LEADTIME", line 14 ORA-04088: error during execution of trigger
'YELLOWCOM.TRG_ORDERFORM_LEADTIME' ORA-06512: at "YELLOWCOM.INSERT_ORDER_FORM", line 14 ORA-06512: at line 11

OPERATION 03

New order

Select the client, dates and requested services.

Client

Andrea Rossi · 123456789

Wedding date

24/09/2026

Filing date

23/09/2026

Requested services

☒ Autumn vineyard wedding banquet

6500.0 - 14000.0 · lead time 120 days

Figure 15: Web Application - Order Form Lead Time Error